

Deep Research: Avatrada 57 Topic 003

Engineering Report: GEX Normalization Pipeline (Z-Score)

Executive Summary

This report provides a detailed engineering analysis of the GEX Normalization system component, as specified in the source document. The primary objective is to convert raw Dollar Gamma Exposure (GEX) into a standardized Z-Score signal, $GEX_{\{norm\}}$, to ensure comparability across different assets and market conditions. The analysis deconstructs the statistical methodology, provides robust implementation strategies in Python (Pandas) and SQL, and conducts a critical review of potential failure modes, edge cases, and optimizations.

The core task is to implement a rolling Z-score calculation: $(Current_GEX - Mean_GEX_{20d}) / StdDev_GEX_{20d}$. This report also addresses the ambiguity in the source document regarding the role of Average Daily Dollar Volume (ADDV), presenting it as a potential alternative normalization factor and clarifying its calculation.

1. Technical Deconstruction

The system's goal is to transform a raw, nominal value (Dollar GEX) into a relative, standardized signal. This is a common practice in quantitative finance to make signals comparable over time and across different underlying instruments.

1.1 Core Components & Formulas

a) Raw Dollar GEX: This is the input signal. Gamma Exposure (GEX) typically represents the total gamma of outstanding options for a given underlying, weighted by open interest. Dollar GEX translates this into a nominal dollar value,

indicating the dollar amount of the underlying that market makers would need to buy or sell to remain delta-neutral for a 1% move in the underlying's price. *

Formula: `Dollar_GEX ≈ GEX_per_share * Spot_Price * 100` (assuming 100 shares per option contract).

b) Z-Score Normalization: The Z-score is a statistical measure that describes a value's relationship to the mean of a group of values. It is measured in terms of standard deviations from the mean. A Z-score of 0 indicates the value is identical to the mean, while a Z-score of 1.0 indicates a value that is one standard deviation above the mean.

- **Mathematical Definition:** `latex Z = \frac{X - \mu}{\sigma}`

- **In this Context:**

- `X` = `Current_GEX` : The raw Dollar GEX for the current observation period (e.g., daily).
- `μ` = `Mean_GEX_20d` : The 20-day rolling simple moving average (SMA) of raw Dollar GEX.
- `σ` = `StdDev_GEX_20d` : The 20-day rolling sample standard deviation of raw Dollar GEX.

c) Rolling Window: A 20-day rolling window is specified. This means that for any given day `T`, the mean and standard deviation are calculated using data from day `T` back to day `T-19`. This makes the normalization adaptive to recent market behavior.

d) Average Daily Dollar Volume (ADDV): The source context mentions using a 20-day ADDV as a baseline. While the Z-score formula provided in the prompt does *not* directly use ADDV as a denominator, its calculation is a related and important concept for market context. * **Daily Dollar Volume:** `Daily_Dollar_Volume = Daily_Volume * Daily_VWAP` (or `Daily_Volume * Daily_Close_Price` as a simpler proxy). * **20-Day ADDV:** The 20-day simple moving average of the `Daily_Dollar_Volume`.

e) Outlier Handling: The prompt requires a mechanism to handle outliers. Extreme GEX values, often occurring during major market events (e.g., "volmageddon," large OPEX), can disproportionately skew the rolling mean and standard deviation, making the Z-score less reliable for typical periods. A

common technique is **Winsorization**, where extreme values are capped at a certain percentile. For example, clipping all values below the 5th percentile and above the 95th percentile.

2. Implementation Strategy

This section provides concrete code for building the normalization pipeline.

2.1 Data Schema

We assume a time-series dataset with the following columns: * `trade_date`: The date of the observation. * `ticker`: The underlying asset symbol. * `raw_dollar_gex`: The raw Dollar GEX value for that day. * `close_price`: The closing price of the underlying. * `volume`: The trading volume of the underlying.

2.2 Python (Pandas) Implementation

Pandas is exceptionally well-suited for this type of rolling time-series analysis. The implementation below includes robust outlier handling.

```
import pandas as pd
import numpy as np

def calculate_gex_zscore(df: pd.DataFrame, window: int = 20,
outlier_clip_level: float = 0.05) -> pd.DataFrame:
    """
    Calculates a rolling Z-score for raw_dollar_gex with outlier handling.

    Args:
        df (pd.DataFrame): DataFrame with 'trade_date', 'raw_dollar_gex'.
                           Must be sorted by date.
        window (int): The rolling window period.
        outlier_clip_level (float): The percentile for clipping (e.g., 0.05 for
        5th/95th).

    Returns:
        pd.DataFrame: Original DataFrame with new columns for the normalized
```

```

signal.
"""
# Ensure data is sorted by date
df = df.sort_values('trade_date').copy()

# --- Step 1: Outlier Handling (Winsorization) ---
# Calculate rolling quantiles for clipping
lower_bound =
df['raw_dollar_gex'].rolling(window=window).quantile(outlier_clip_level)
upper_bound = df['raw_dollar_gex'].rolling(window=window).quantile(1 -
outlier_clip_level)

# Clip the raw GEX values based on the rolling bounds
# .bfill() handles the initial NaN values in the bounds
df['gex_clipped'] = df['raw_dollar_gex'].clip(
    lower=lower_bound.bfill(),
    upper=upper_bound.bfill()
)

# --- Step 2: Calculate Rolling Statistics on the Clipped Data ---
rolling_mean = df['gex_clipped'].rolling(window=window).mean()
rolling_std = df['gex_clipped'].rolling(window=window).std()

# --- Step 3: Calculate the Z-Score ---
# Add a small epsilon to the denominator to prevent division by zero
epsilon = 1e-9
df['gex_zscore'] = (df['gex_clipped'] - rolling_mean) / (rolling_std +
epsilon)

# --- (Optional) Calculate 20-Day ADDV for context ---
df['dollar_volume'] = df['close_price'] * df['volume']
df['addv_20d'] = df['dollar_volume'].rolling(window=window).mean()

return df

# Example Usage:
# Assume `data` is a DataFrame with the required columns
# data_with_signal = calculate_gex_zscore(data)
# print(data_with_signal.tail())

```

2.3 SQL Implementation (Window Functions)

For data residing in a modern data warehouse (like PostgreSQL, BigQuery, or Snowflake), SQL window functions are highly efficient.

This example uses Common Table Expressions (CTEs) for clarity. Outlier handling is more complex in SQL; this example focuses on the core Z-score calculation. A pre-processing step or a more complex query with `PERCENTILE_CONT` would be needed for robust Winsorization.

```
-- Assumes a table named 'daily_market_data' with columns:
-- trade_date, ticker, raw_dollar_gex

WITH RollingStats AS (
  SELECT
    trade_date,
    ticker,
    raw_dollar_gex,
    -- Calculate 20-day rolling average of GEX
    AVG(raw_dollar_gex) OVER (
      PARTITION BY ticker
      ORDER BY trade_date
      ROWS BETWEEN 19 PRECEDING AND CURRENT ROW
    ) AS mean_gex_20d,

    -- Calculate 20-day rolling sample standard deviation of GEX
    STDDEV_SAMP(raw_dollar_gex) OVER (
      PARTITION BY ticker
      ORDER BY trade_date
      ROWS BETWEEN 19 PRECEDING AND CURRENT ROW
    ) AS stddev_gex_20d
  FROM
    daily_market_data
)
SELECT
  trade_date,
  ticker,
  raw_dollar_gex,
  mean_gex_20d,
```

```
stddev_gex_20d,  
-- Calculate the Z-Score, handling potential division by zero  
CASE  
    WHEN stddev_gex_20d IS NULL OR stddev_gex_20d = 0 THEN 0  
    ELSE (raw_dollar_gex - mean_gex_20d) / stddev_gex_20d  
END AS gex_zscore  
FROM  
    RollingStats  
ORDER BY  
    ticker,  
    trade_date;
```

3. Critical Analysis

3.1 Potential Failure Modes & Edge Cases

1. **Insufficient Data History:** For the first 19 days of any given time series, the rolling window is incomplete, resulting in `NULL` or `NaN` values.
 - **Mitigation:** The system must decide how to handle this initialization period. Options include:
 - Accepting `NULL` values and only trading the signal after 20 days.
 - Using a shorter, expanding window until the 20-day period is reached.
 - Backfilling with historical data if available.
2. **Zero Standard Deviation:** During periods of extremely low volatility or stable GEX positioning, the 20-day standard deviation can approach zero.
 - **Failure:** This leads to a division-by-zero error, producing `Infinity` or `NaN`.
 - **Mitigation:** As implemented in the Python snippet, add a small constant (epsilon) to the denominator. The SQL example uses a `CASE` statement to return 0 in this scenario. The choice of returning 0 is logical, as a zero standard deviation implies the current value is exactly at the mean.

3. **Outlier Handling Strategy:** The choice of clipping percentile (e.g., 5%/95% vs. 1%/99%) is a critical parameter.

- **Risk:** Overly aggressive clipping can dampen the signal's responsiveness to genuinely significant market regime shifts. Insufficient clipping fails to stabilize the mean/std dev calculations.
- **Mitigation:** This parameter should be calibrated and backtested based on the specific asset and desired signal behavior.

4. **Lookahead Bias:** A common implementation error is to use data from the future to calculate statistics for the present.

- **Mitigation:** The provided Pandas `.rolling()` and SQL `ROWS BETWEEN 19 PRECEDING AND CURRENT ROW` implementations correctly use a lagging window, preventing lookahead bias. This must be strictly enforced in any custom implementation.

3.2 Optimizations & Alternative Approaches

1. **Exponentially Weighted Moving Average (EWMA):** The specification calls for a simple moving average (SMA), which weights all 20 data points equally. An EWMA could be superior.

- **Advantage:** EWMA gives more weight to recent data, making the mean and standard deviation more responsive to recent changes in market dynamics.
- **Implementation:** In Pandas, this is a simple change from `.rolling(20).mean()` to `.ewm(span=20).mean()`.

2. **Alternative Normalization: Scaling by ADDV:** The source document's mention of ADDV suggests an alternative normalization scheme that was potentially conflated with the Z-score. This method measures GEX relative to market liquidity.

- **Formula:**
$$GEX_{\text{norm_addv}} = \frac{\text{RawDollarGEX}}{\text{ADDV}_{\{20d\}}}$$
- **Interpretation:** This signal represents the size of the dealer hedging requirement as a fraction of the typical daily traded value. A high

value might suggest that dealer hedging flows could have a more significant impact on price action.

- **Comparison:**

- **Z-Score:** Measures how *unusual* the current GEX level is compared to its *own recent history*. It is unitless.
- **ADDV-Scaled:** Measures the *magnitude* of GEX relative to *market activity*. It is also unitless (dollar/dollar).
- **Conclusion:** Both signals are valid but answer different questions. The Z-score is better for identifying statistical extremes, while the ADDV-scaled version is better for gauging market impact potential.

3. **Computational Performance:** For extremely large datasets (e.g., tick-level GEX), standard rolling calculations can be a bottleneck.

- **Optimization:** Libraries like `Numba` or `Polars` in the Python ecosystem can significantly accelerate these numerical computations. For database-centric workflows, ensuring proper indexing on `(ticker, trade_date)` is critical for performance.